

List of AI Engineer Interview questions + concepts:

♦ **LLM Fundamentals**

- What is tokenization, and how does it affect generation?
- How do embeddings really work?
- What's the role of attention, positional encoding?
- What changes during fine-tuning? (optimizers, schedulers, layer freezing)
- LoRA vs QLoRA vs full fine-tune - tradeoffs?

♦ **Prompting & Context Engineering**

- Few-shot vs zero-shot - which works better where?
- How do you design system prompts that are robust across users?
- How do you make output deterministic?
- How do you track, version, and backfill changing context?
- How do you build/maintain the memory?

♦ **RAG Systems**

- What's your chunking strategy - by length, semantics, or structure?
- How do you choose a vector DB (Chroma, Pinecone, OpenSearch...)?
- Can you update or backfill embeddings with zero downtime?
- How do you evaluate retrieval quality (precision@k, reranking, citation)?

♦ **MLOps & LLMops**

- Sketch a pipeline: from raw data → model → serving → feedback
- How would you monitor performance drift or hallucinations?
- How do you log prompts and outputs for debugging and auditing?
- CI/CD for LLM workflows - what's different from ML?

♦ **Cost & Latency Tradeoffs**

- How do you reduce token usage?
- When should you quantize a model?
- What's your batching + caching strategy to reduce latency?
- When to use hosted APIs vs open-source models?

♦ **System Design Thinking**

- How do you make an AI system more deterministic and less brittle?
- What fallback do you use if the LLM fails mid-task?
- Can you solve this without an LLM or vector DB?
- What's the right database for this task - SQL, NoSQL, or vector?

♦ **Real-World Scenarios:**

- ① What happens if your embedding model changes - how do you migrate safely?
- ② How would you fine-tune a model on user behavior and deploy it?
- ③ How would you make this system cheaper without killing quality?
- ④ Can you walk me through a debugging session for incorrect LLM outputs?